

processes as a starting point to ns-3 simulation. A socket acts as an end point to a communication channel. In order to set up communication between two processes, there should be two sockets—one at the sender's side and another at the receiver's side. The behaviour of these two sockets is different. Often, one behaves like a server socket and the other like a client socket. The client tries to connect to the server to obtain data. For practical implementation, socket APIs provided by the operating system are used. The most commonly used socket APIs are the Berkeley sockets (BSD sockets), which essentially are a computing library for Internet sockets and UNIX domain sockets used for inter-process communication. These APIs were originally developed for C but can be used with C++ also, without any changes.

Since most communication nowadays involves the Internet, let us only discuss the Internet socket. It is characterised by its address and the protocol used. The address of an Internet socket consists of the IP address and the port number, thus involving the transport layer and the network layer. Another important aspect is the underlying transport protocol used; the choices include TCP and UDP.

Communication involving sockets can be broadly classified into two types—reliable communication (client-server) and unreliable communication (peer-to-peer). In reliable communication involving TCP, a virtual circuit is established, which guarantees packet transmission. In unreliable communication involving UDP, a datagram based communication model is followed, which only provides the unordered delivery of packets. There are a number of socket APIs (system-defined functions provided by various header files), which help us realise TCP and UDP based communication. First, let us discuss the APIs used commonly by both TCP and UDP connections.

Socket APIs common to TCP and UDP connections

socket: The *socket* API is used to create a socket of a given domain, type and protocol. In case of a TCP connection, both client and server use this function. For a UDP connection, both the peers (nodes) use this function.

```
int socket (int domain, int type, int protocol);
```

The domain is AF_INET (IPv4 only) or AF_INET6 (IPv6 or IPv4). The type is SOCK_STREAM for TCP and SOCK_DGRAM for UDP. The protocol is usually set as 0 so that the type defines the connection of the domain.

bind: The *bind* API gives the address of the socket on the server side. It binds a particular socket with an address to which other processes can connect. In the Internet protocol, the address is a combination of the IP address and port address. The *bind* API is only used at the server side in TCP and at the receiver side in UDP.

```
int bind ( int sid , struct sockaddr *addrPtr , int len );
```

Here *sid* is the socket ID, *addrPtr* points to the address structure and *len* is the size of the pointer *addrPtr*.

close: The *close* API is used to close a socket and terminate the connection. This results in freeing all the kernel data structures previously allocated.

```
int close( int sockfd );
```

Socket APIs specific to the TCP connection

send: The *send* API is used to send a message over a stream. It returns the number of bytes sent or -1 in case of a failure.

```
int send (int sid, const char *bufferPtr, int len, int flag);
```

Here *bufferPtr* contains the message to be sent and the flag is set as 0, by default.

recv: The *recv* API is used to receive up to *len* bytes in *bufferPtr*, and it returns the number of bytes received or -1 in case of failure.

```
int recv (int sid, const char *bufferPtr, int len, int flag);
```

Socket APIs that are specific to the TCP server

listen: The *listen* API fixes the maximum number of connections the server can accept. It returns 0 in case of success or -1 in case of failure.

```
int listen (int sid, int size);
```

Here, *size* sets the maximum number of connections the server can accept.

accept: The *accept* API connects a specific client to the server. It waits for an incoming *connect* request from the client, and returns the socket ID and address of the client connected.

```
int accept (int sid, struct sockaddr *addrPtr, int *lenPtr);
```

Here, the *accept* API creates a new socket for the client whose connection it has accepted.

Socket APIs specific to TCP clients

connect: The *connect* API is used by the client process to specify the address of the server to connect with.

```
int connect (int sid, struct sockaddr *addrPtr, int len);
```

Here, *addrPtr* specifies the server address to connect with, and the API returns 0 in case of success and -1 in case of failure.

Socket APIs specific to UDP connections

sendto: The *sendto* API is the UDP counterpart of the TCP API *send*. It additionally specifies the address to which